



ONE SEMESTER PROJECT

IMPLEMENTATION PLAN DOCUMENT

24/06590

KALOKI CARRICK MWELII

BDS 3106

BACHELOR OF SCIENCE IN DATA SCIENCE

SUPERVISOR - LISPER KENDI

PROJECT TITLE : SMART BANKLYTICS

An Intelligent Data – Driven Bank Management and Customer Analytics System

Table of Contents

1. INTRODUCTION	3
1.1 Purpose	3
1.2 Scope	3
1.3 Implementation Overview	3
2.0 IMPLEMENTATION SCHEDULE.....	3
3.0 INSTALLATION AND CONVERSION PLANS	4
3.1 Software Installation Plan	4
3.2 Hardware Installation Plan	5
3.3 Activities of Conversion.....	5
4.0 SYSTEM CONVERSION STRATEGY	6
5.0 TRAINING PLAN	6
5.1 Training Methods	6
6.0 SOFTWARE MAINTENANCE PLAN	6
7.0 CHANGE MANAGEMENT PLAN	7
PHASE 1: DATABASE DESIGN AND INITIALIZATION	7
PHASE 2: USER AUTHENTICATION MODULE	8
PHASE 3: CUSTOMER ACCOUNT MANAGEMENT MODULE.....	8
PHASE 4: TRANSACTION PROCESSING MODULE	9
PHASE 5: FRAUD DETECTION MODULE.....	10
PHASE 6: ADMINISTRATOR MANAGEMENT MODULE	10
PHASE 7: ANALYTICS AND KPI DASHBOARD.....	11
PHASE 8: PASSWORD RECOVERY MODULE	12
PHASE 9: PDF STATEMENT GENERATION	12
PHASE 10: SYSTEM INTEGRATION AND TESTING	13
9.0 CHALLENGES AND SOLUTIONS.....	13
10.0 CONCLUSION.....	14

1. INTRODUCTION

1.1 Purpose

This Implementation Plan document outlines the structured approach taken to develop and deploy the Smart Banklytics system. It describes the phases, tasks, tools, timelines, and responsibilities involved in transforming the system design into a fully functional web-based banking and analytics platform.

1.2 Scope

The implementation covers all system modules including user authentication, account management, transaction processing, fraud detection, analytics dashboard, password recovery, and PDF statement generation. The system was developed using Python (Flask), SQLite, HTML, CSS, JavaScript, and the fpdf library.

1.3 Implementation Overview

Smart Banklytics was developed following an **Agile development methodology**, enabling incremental building, testing, and refinement of each module. Development was carried out in structured phases, with each phase producing a working component of the system before moving to the next.

2.0 IMPLEMENTATION SCHEDULE

Phase	Activity	Duration
Phase 1	Database Design and Initialization	Week 1
Phase 2	User Authentication Module	Week 2
Phase 3	Customer Account Management	Week 2
Phase 4	Transaction Processing Module	Week 3
Phase 5	Fraud Detection Module	Week 4
Phase 6	Administrator Management Module	Week 5

Phase 7	Analytics and KPI Dashboard	Week 6
Phase 8	Password Recovery Module	Week 6
Phase 9	PDF Statement Generation	Week 7
Phase 10	System Integration and Testing	Week 8

3.0 INSTALLATION AND CONVERSION PLANS

3.1 Software Installation Plan

The following software components are required for system deployment:

- Python 3.x installed on the host machine
- Flask framework installed via pip
- Required libraries: fpdf, pandas, numpy, matplotlib
- SQLite database initialized (bank.db)
- Web browsers: Google Chrome or Mozilla Firefox

Before implementation began, the following environment was configured:

Setup Steps Performed

1. Python 3.x installed on the development machine.
2. Flask and required libraries installed via pip: pip install flask fpdf pandas numpy matplotlib.
3. Project folder structure created with the following layout:

```
smart_banklytics/  
|  
├─ app.py  
├─ bank.db  
├─ static/  
|   └─ images/  
|       └─ logo.png  
└─ templates/  
    ├─ login.html  
    ├─ dashboard.html  
    ├─ admin.html  
    ├─ deposit.html  
    ├─ withdraw.html  
    ├─ transfer.html  
    ├─ signup.html  
    ├─ forgot_password.html  
    ├─ view_users.html  
    └─ manage_user.html
```

4. SQLite database initialized by navigating to /init-db route, which created the users, transactions, and fraud_alerts tables.
5. A default admin account was manually inserted into the database to enable initial system access.

3.2 Hardware Installation Plan

The system requires the following hardware:

- Server or local machine capable of running Python and Flask
- Minimum 4GB RAM and modern processor
- Client devices (PCs or laptops) with web browsers
- Stable internet or local network connection

3.3 Activities of Conversion

- System setup and configuration
- Database initialization and population
- Creation of initial admin account
- Testing system functionality before deployment
- Deployment of system to users

4.0 SYSTEM CONVERSION STRATEGY

A Direct Conversion strategy is adopted. The new Smart Banklytics system replaces any previous manual or basic systems immediately upon deployment. This approach is suitable because the system is newly developed and does not require gradual transition from an existing automated system.

5.0 TRAINING PLAN

Training is conducted to ensure effective system usage by all stakeholders.

5.1 Training Methods

- User manuals and documentation
- Demonstration sessions
- Hands-on practical training

Target Users:

- Administrators: System management and monitoring
 - Customers: Basic account operations and navigation
-

6.0 SOFTWARE MAINTENANCE PLAN

The system will be maintained through:

- Regular bug fixing and updates
 - Performance monitoring and optimization
 - Database maintenance and backups
 - Security updates and vulnerability patching
-

7.0 CHANGE MANAGEMENT PLAN

Changes to the system will be managed through a structured process:

- Identification and documentation of required changes
- Approval from system supervisor or stakeholder
- Implementation and testing of changes
- Version control using Git
- Communication of updates to users

8.0 IMPLEMENTATION PHASES

PHASE 1: DATABASE DESIGN AND INITIALIZATION

Objective: Design and create the relational database schema that supports all system operations.

Tasks Completed:

- Designed the users table with fields: id, username, password, first_name, last_name, id_number, account_number, account_type, balance, role, and status.
- Designed the transactions table with fields: id, account_number, type, amount, date, from_account, and to_account.
- Designed the fraud_alerts table with fields: id, user_id, account_number, reason, amount, date, status, action, from_account, to_account, and type.
- Implemented the init_db() route in app.py to programmatically initialize all three tables using CREATE TABLE IF NOT EXISTS statements.
- Tested database creation by running the Flask application and visiting the /init-db endpoint.

Outcome: A fully initialized SQLite database ready to support all system modules.

PHASE 2: USER AUTHENTICATION MODULE

Objective: Implement secure login, session management, and role-based redirection.

Tasks Completed:

- Created the /login route handling both GET and POST requests.
- Implemented credential validation by querying the users table against submitted username and password.
- Configured Flask session management using app.secret_key to maintain user sessions across requests.
- Implemented role-based redirection: customers are redirected to the customer dashboard, administrators to the admin dashboard.
- Created the login.html template with a login form and error message display.
- Added session checks across all protected routes to redirect unauthenticated users to the login page.

Outcome: Secure login system with session-based access control and role-based dashboard routing.

PHASE 3: CUSTOMER ACCOUNT MANAGEMENT MODULE

Objective: Enable customers to view account details, balances, transaction history, and fraud alerts.

Tasks Completed:

- Implemented the /dashboard route to fetch and display the logged-in user's account information.
- Queried the transactions table to retrieve and display the user's full transaction history ordered by most recent.
- Queried the fraud_alerts table to display any fraud alerts linked to the user's account.
- Created the dashboard.html template displaying account details, balance, transaction history, and alert notifications.

Outcome: A fully functional customer dashboard with real-time account and transaction information.

PHASE 4: TRANSACTION PROCESSING MODULE

Objective: Implement deposit, withdrawal, and transfer functionality with full validation and transaction logging.

Tasks Completed:

Deposit:

- Implemented the /deposit route accepting account number and amount.
- Added input validation for zero, negative, and non-numeric amounts.
- Integrated fraud detection check before processing.
- Updated user balance and inserted a deposit record into the transactions table upon successful processing.

Withdrawal:

- Implemented the /withdraw route for logged-in customers.
- Added balance sufficiency check before processing withdrawals.
- Blocked withdrawals for suspended accounts.
- Integrated fraud detection check before processing.
- Updated balance and logged the withdrawal transaction upon success.

Transfer:

- Implemented the /transfer route for logged-in customers.
- Validated that sender and receiver accounts are different.
- Verified receiver account existence before processing.
- Checked sender balance sufficiency.
- Blocked transfers for suspended accounts.
- Integrated fraud detection check before processing.
- Performed atomic balance updates for both sender and receiver.
- Logged both transfer_out and transfer_in records in the transactions table.

Outcome: Fully functional and secure transaction processing with comprehensive input validation and fraud integration.

PHASE 5: FRAUD DETECTION MODULE

Objective: Implement automated fraud detection rules that monitor transactions and trigger alerts.

Tasks Completed:

- Implemented the `check_fraud()` function in `app.py`, called before every transaction is processed.
- Implemented **Rule 1 – Large Transaction Detection:** Any single transaction exceeding KES 100,000 triggers a fraud alert and automatically suspends the account.
- Implemented **Rule 2 – Rapid Transaction Detection:** Three or more transactions by the same account within a 5-minute window triggers a fraud alert and suspends the account.
- Configured the function to insert a detailed record into the `fraud_alerts` table capturing: user ID, account number, reason, amount, transaction type, source account, and destination account.
- Integrated `check_fraud()` into the deposit, withdrawal, and transfer routes.

Outcome: Automated fraud detection system that monitors all transactions in real time and responds immediately to suspicious activity.

PHASE 6: ADMINISTRATOR MANAGEMENT MODULE

Objective: Provide administrators with tools to manage users, review fraud alerts, and monitor system activity.

Tasks Completed:

- Implemented the `/admin` route to serve the admin dashboard.
- Implemented the `/signup` route allowing administrators to create new customer accounts with auto-generated account numbers starting from 301120060001.
- Implemented the `/view_users` route displaying all registered users.

- Implemented the /manage/<account_number> route for detailed user management including fraud alert review.
- Implemented the /toggle_status/<account_number> route to suspend or activate user accounts.
- Implemented fraud alert review actions: **Legitimate** (processes the blocked transaction and activates the account), **Investigate** (flags the alert for further review), and **Freeze** (confirms account suspension).
- Implemented the customer appeal feature via the /appeal_alert route, allowing users to contest fraud alerts.

Outcome: A comprehensive admin management interface with full user control and fraud response capabilities.

PHASE 7: ANALYTICS AND KPI DASHBOARD

Objective: Build an analytics dashboard that displays key banking performance metrics and transaction trend visualizations.

Tasks Completed:

- Computed the following KPIs from the database within the /admin route:
 - Total number of registered users
 - Total number of transactions
 - Total deposits, withdrawals, and transfers
 - Total bank balance (sum of all user balances)
- Aggregated transaction data by date to generate time-series data for trend charts.
- Computed cumulative balance values over time.
- Retrieved account type distribution data for pie/bar chart visualization.
- Passed all computed data to admin.html for rendering using Chart.js.

Outcome: A fully functional analytics dashboard with live KPI metrics and interactive transaction trend visualizations.

PHASE 8: PASSWORD RECOVERY MODULE

Objective: Allow users to securely reset forgotten passwords through multi-field identity verification.

Tasks Completed:

- Implemented the /forgot-password route handling both GET and POST requests.
- Implemented identity verification by matching username, account number, and national ID number simultaneously.
- Added password confirmation check to ensure both password fields match before updating.
- Updated the password field in the users table upon successful verification.
- Created the forgot_password.html template with the recovery form and success/error message display.

Outcome: A secure password recovery system that verifies user identity using multiple credentials before allowing a password reset.

PHASE 9: PDF STATEMENT GENERATION

Objective: Enable customers to download their transaction history as a formatted PDF bank statement.

Tasks Completed:

- Implemented the /download_pdf route using the fpdf library.
- Added filter options: past 30 days, past 14 days, and custom date range.
- Added transaction type filtering (deposit, withdrawal, transfer).
- Built the PDF dynamically with the bank logo, customer name, account number, balance, date range header, and a formatted transaction table.
- Generated the PDF in memory using io.BytesIO and delivered it to the browser as a downloadable file using Flask's send_file.

Outcome: A fully functional PDF statement generator with flexible filtering options and professional formatting.

PHASE 10: SYSTEM INTEGRATION AND TESTING

Objective: Integrate all modules into a unified system and conduct comprehensive functional testing.

Tasks Completed:

- Integrated all routes and modules within a single app.py file.
- Conducted end-to-end functional testing covering all 53 test cases as documented in the Test Plan.
- Verified correct data flow between the frontend templates, Flask backend, and SQLite database.
- Confirmed fraud detection triggers correctly across deposit, withdrawal, and transfer operations.
- Validated admin fraud review actions process transactions and update account statuses correctly.
- Tested PDF generation with all filter combinations.
- Confirmed analytics KPIs reflect accurate database values.
- Resolved identified bugs and inconsistencies during testing.

Outcome: A fully integrated, tested, and stable Smart Banklytics system ready for deployment.

9.0 CHALLENGES AND SOLUTIONS

Challenge	Solution
Fraud detection needed to run before transaction commit	Implemented <code>check_fraud()</code> as a pre-transaction function called before any balance update or transaction log entry
Transfer required atomic updates for both accounts	Used a single database connection with commit after both UPDATE statements to ensure atomicity
PDF generation needed in-memory delivery without saving to disk	Used <code>io.BytesIO</code> to generate and stream the PDF directly to the browser

Challenge	Solution
Admin needed to process blocked transactions post-review	Implemented the "legitimate" action in the fraud review handler to reconstruct and execute the original transaction
Session security for protected routes	Added session.get('username') checks at the start of every protected route with redirect to login on failure

10.0 CONCLUSION

The Smart Banklytics system was successfully implemented across ten structured phases following an Agile development approach. All planned modules were developed, integrated, and tested. The system delivers a secure, functional, and intelligent banking platform that combines transaction management, real-time fraud detection, analytics dashboards, and automated reporting in a unified web-based application.